



**GUJARAT TECHNOLOGICAL
UNIVERSITY**
Shree Swami Atmanand Saraswati Institute of
Technology



A

Micro Project Report on

College Placement Data Analyzer

Under subject of

Python for Data Science (BE05000231)

B.E. 3rd Year, Semester - V

COMPUTER ENGINEERING

Under the Guidance of

Prof. Vipul S. Kania

Submitted by group:

Sr. No.	Name	Enrollment No.
1	Dudhalva Hardik	240760107045
2	Beldar Chetan	250763107002
3	Chauhan Parth	250763107006
4	Sadrani Aditya	240760107001
5	Avaiya Het	240760107008
6	Jeel Goyani	240760107062

Academic Year: 2026-2027

SHREE SWAMI ATMANAND SARASWATI INSTITUTE OF
TECHNOLOGY

Opp. Kapodra Police Station, Varachha Road, Surat - 395006

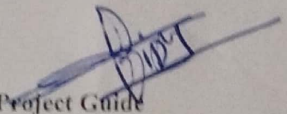


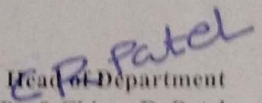
CERTIFICATE

This is to certify that the project entitled "College Placement Data Analyzer" has been prepared by Chetan Beldar (250763107002), Chauhan Parth (250763107006), Dhundhalva Hardik (240760107045), Saradani Aditya (240760107001), Avaiya Het (240760107008), and Jeel Goyani (240760107062) in the subject of Project Management (BE05000481) under my guidance, in partial fulfilment of the degree of Bachelor of Engineering in Computer Engineering (5th Semester) of Gujarat Technological University, Ahmedabad, during the academic year 2026-2027.

Date:

Place: SSASIT, Surat


Project Guide
Prof. Vipul S. Kania


Head of Department
Prof. Chirag R. Patel

Acknowledgement

*We would like to express our sincere thanks to everyone who helped us in completing our micro project "College Placement Data Analyzer". First of all, we are very grateful to our project guide **Prof. Vipul S. Kania** for his continuous guidance, valuable suggestions and patience throughout the work, because of which we could understand the concepts of Python for Data Science properly and finish this project on time. We also thank **Prof. Chirag Patel**, Head of the Computer Engineering Department, for giving us the chance and the facilities to do this work, and we are thankful to our Principal and all the faculty members of **Shree Swami Atmanand Saraswati Institute of Technology** for their support and encouragement. We thank **Gujarat Technological University** for including such a practical subject in the curriculum, which helped us to learn how to work with real data using Python, Pandas, NumPy, Matplotlib, Seaborn, SciPy and Scikit-learn. We also want to thank all the team members, who worked together, shared the work equally, and helped each other whenever someone got stuck with errors in the code or with the report. Finally, we are thankful to our parents and friends for their motivation and support during the whole semester.*

Abstract

Every college wants to know how many of its students will get placed and which factors matter most in the campus placement process. This project, College Placement Data Analyzer, is a Python based data analysis work done on a dataset of 500 students. The dataset has twelve columns such as CGPA, backlogs, projects, internship, technical skills, aptitude score, communication score and certifications. First the data was loaded and checked for missing values, duplicate rows and outliers using the IQR rule, and it was found that the data is already clean. After that, Min-Max normalization was applied on four numeric columns. Since the dataset has no placement column, we created our own composite score out of 100 and an eligibility rule (CGPA of at least 6.0 and not more than one backlog) to decide the placement status, and a rough salary estimate for placed students.

Descriptive statistics, skewness, kurtosis and a one sample t-test were calculated for CGPA, and department wise placement percentages were found. Six graphs were drawn to understand the data visually. In the end a Random Forest classifier was trained, which gave 97% accuracy on the test data, with CGPA and backlogs as the most important features. As the placement label was made from the same columns, this accuracy mainly shows that the model learned our rule, and this point is discussed in the limitations. The project shows how the basic tools of Python can be used step by step to clean, analyze, visualize and model student placement data.

Keywords: Placement Analysis, Pandas, Data Cleaning, Normalization, t-test, Random Forest, Data Visualization

Table of Contents

Abstract.....	i
Table of Contents	ii
List of Figures.....	iv
List of Tables	v
1. Introduction.....	1
1.1 Background	1
1.2 Motivation	1
1.3 Scope of the Project.....	1
2. Problem Statement and Objectives	2
2.1 Problem Statement	2
2.2 Objectives.....	2
3. Literature Survey.....	3
3.1 Existing Approaches	3
3.2 Positioning of This Project.....	3
4. Dataset Description	4
4.1 About the Dataset	4
4.2 Column Description	4
4.3 Descriptive Summary	5
5. Tools and Technologies.....	6
5.1 Software and Libraries	6
6. System Design and Methodology	7
6.1 Overall Workflow	7
6.2 Placement Score Rule.....	7
7. Implementation	9
7.1 Importing Libraries	9
7.2 Loading the Dataset.....	9
7.3 Data Cleaning and Checking.....	10
7.4 Normalization.....	11
7.5 Placement Score, Status and Salary	11
7.6 Statistics and Hypothesis Test	12
7.7 Data Visualization	13
7.8 Random Forest Model.....	13
8. Results and Discussion.....	15

8.1 CGPA Distribution.....	15
8.2 Placement Result.....	15
8.3 CGPA and Placement.....	16
8.4 Department-wise Placement.....	16
8.5 Correlation Analysis.....	17
8.6 Aptitude and Communication	18
8.7 Effect of Internship.....	19
8.8 Salary Estimate.....	19
8.9 Model Performance.....	20
9. Conclusion, Limitations and Future Scope	22
9.1 Conclusion.....	22
9.2 Limitations	22
9.3 Future Scope.....	22
References.....	23

List of Figures

Figure 1: Number of students by gender and department	4
Figure 2: Workflow of the College Placement Data Analyzer	7
Figure 3: Distribution of CGPA	15
Figure 4: Placed and Not Placed students	16
Figure 5: CGPA vs placement status	16
Figure 6: Placement by department.....	17
Figure 7: Correlation heatmap.....	18
Figure 8: Aptitude score vs communication score	19
Figure 9: Internship vs placement	19
Figure 10: Confusion matrix of the Random Forest model	20
Figure 11: Feature importance	21

List of Tables

Table 1: Comparison of existing approaches	3
Table 2: Columns of the dataset.....	4
Table 3: Descriptive statistics of numeric columns	5
Table 4: Tools and libraries used	6
Table 5: Weights used in the composite score	8
Table 6: Department-wise placement.....	17
Table 7: Classification report on test data	20
Table 8: Feature importance.....	21

1. Introduction

1.1 Background

Campus placement is one of the most important things for an engineering student and also for the college. Every year companies visit colleges and select students on the basis of academic performance, technical knowledge, aptitude and communication skills. Colleges keep a lot of data about their students, like CGPA, backlogs, projects and internships, but most of the time this data is just stored in Excel sheets and nobody analyzes it properly.

Python has become a very popular language for data science because of its libraries such as Pandas, NumPy, Matplotlib, Seaborn, SciPy and Scikit-learn. With these libraries, a large amount of data can be loaded, cleaned, studied and even used to build a prediction model with only a few lines of code.

1.2 Motivation

As students of third year we also face the same situation, so we were curious to know which factors really help a student to get placed. Is CGPA the only thing that matters, or do internships, technical skills and aptitude also play a role? To find answers to such questions with actual numbers and graphs, we decided to make this micro project for the subject Python for Data Science.

1.3 Scope of the Project

The project works on one CSV file containing the records of 500 students from six departments. It covers the complete data science workflow: data loading, cleaning, normalization, creating the placement status, statistical analysis, visualization and a simple machine learning model. The project is done in a Jupyter notebook and runs completely on a local computer.

2. Problem Statement and Objectives

2.1 Problem Statement

Student placement records are generally available in raw form and it is difficult to understand from them which factors affect placement the most. Checking the data manually for errors, calculating percentages department by department and comparing students is slow and mistakes can happen easily. Also, the dataset used in this project does not contain any "placed" or "not placed" column, so it is not possible to do any analysis on placement directly.

Therefore, there is a need for a simple Python based analyzer which can check the quality of the student data, create a placement status using clear rules, show the patterns with statistics and graphs, and also test how well a machine learning model can classify the students.

2.2 Objectives

The main objectives of this project are:

1. To load the placement dataset (CSV file) into a Pandas DataFrame and understand its structure.
2. To check the data for missing values, duplicate records and outliers.
3. To apply Min-Max normalization on important numeric columns.
4. To create a composite score, placement status and salary estimate using a clear rule.
5. To find descriptive statistics, skewness and kurtosis, and to perform a hypothesis test on CGPA.
6. To find the department wise placement percentage.
7. To draw graphs for distribution, comparison and correlation of the data.
8. To train a Random Forest model and find which features are most important.

3. Literature Survey

3.1 Existing Approaches

Before starting the work we studied how placement and student data is normally handled. In general, colleges use one of the approaches given in Table 1. Each of them has some good points and some limits when compared to what we needed for this project.

Table 1: Comparison of existing approaches

Approach	Nature	Strength	Limitation
MS Excel sheets	Spreadsheet software	Easy to use, everyone knows it	Slow for large data, manual formulas, no machine learning
Tableau / Power BI	BI dashboard tools	Very good interactive charts	Paid or limited free version, not meant to teach coding
ML based placement prediction studies	Research work using classifiers	Predict placement from academic and skill data	Need a real placement label, often skip data checking steps
Manual Python scripts	Ad-hoc coding	Full flexibility	No fixed steps, hard to repeat or explain

3.2 Positioning of This Project

Our project sits between simple spreadsheet analysis and big machine learning research. It follows a clear step by step flow in one notebook: check the data, clean it, build the placement label, analyze with statistics, draw graphs and then try a model. It is free, runs locally and is easy to understand for a beginner, which suits the purpose of a micro project.

4. Dataset Description

4.1 About the Dataset

The dataset used is `college_placement_dataset_500.csv`. It has 500 rows and 12 columns, where every row is one student. Six departments are present (Civil, Computer, Electrical, Electronics, IT and Mechanical) with 77 to 99 students each, and the gender is almost equally divided (251 male and 249 female). Memory used by the Dataset is about 47 KB. The data has no missing values and no duplicate rows. The student ages are between 20 and 23 years.

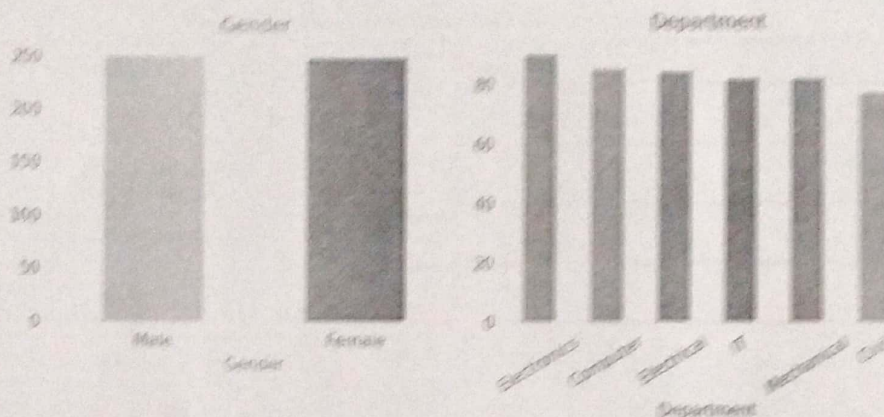


Figure 1: Number of students by gender and department

4.2 Column Description

Table 2: Columns of the dataset

Column	Type	Description
Student_ID	String	Unique ID of student (STU001 to STU500)
Gender	String	Male or Female
Age	Integer	Age of student (20 to 23)
Department	String	Civil, Computer, Electrical, Electronics, IT, Mechanical
CGPA	Float	Cumulative grade point average (5.51 to 9.90)
Backlogs	Integer	Number of pending subjects (0 to 3)
Projects	Integer	Number of projects done (0 to 5)
Internship	String	Whether the student did an internship (Yes / No)
Technical_Skills	Integer	Technical skill rating (1 to 10)
Aptitude_Score	Integer	Aptitude test score (40 to 100)
Communication_Score	Integer	Communication skill score (40 to 100)
Certifications	Integer	Number of certifications (0 to 5)

Three more columns were added by our code later: `Composite_Score` (a score out of 100), `Placement_Status` (Placed / Not Placed) and `Salary_LPA` (estimated package in lakhs per annum for placed students, 0 for others).

4.3 Descriptive Summary

Table 3 gives the summary of the numeric columns. The average CGPA is 7.75 and the middle value is 7.78. Most students have no backlog (361 out of 500), while 25 students have three backlogs.

Table 3: Descriptive statistics of numeric columns

Column	Mean	Std	Min	25%	50%	75%	Max
Age	21.51	1.11	20.00	21.00	22.00	22.00	23.0
CGPA	7.75	1.27	5.51	6.67	7.78	8.83	9.9
Backlogs	0.44	0.82	0.00	0.00	0.00	1.00	3.0
Projects	2.56	1.73	0.00	1.00	3.00	4.00	5.0
Technical_Skills	5.81	2.90	1.00	3.00	6.00	8.00	10.0
Aptitude_Score	69.83	17.56	40.00	55.00	70.00	85.00	100.0
Communication_Score	69.52	17.00	40.00	55.00	69.00	84.00	100.0
Certifications	2.49	1.74	0.00	1.00	2.00	4.00	5.0

5. Tools and Technologies

5.1 Software and Libraries

The whole project is written in Python and executed in a Jupyter notebook. The libraries used and their purpose are listed in Table 4.

Table 4: Tools and libraries used

Tool / Library	Used for
Python 3	Main programming language
Jupyter Notebook	Writing code and seeing output cell by cell
Pandas	Loading CSV, DataFrame operations, group by, value counts
NumPy	Numerical work and np.where conditions
Matplotlib	Drawing and saving the graphs
Seaborn	Histogram, boxplot, count plot, heatmap, scatter plot
SciPy (stats)	Skewness, kurtosis and one sample t-test
Scikit-learn	MinMaxScaler, train_test_split, RandomForestClassifier and metrics

6. System Design and Methodology

6.1 Overall Workflow

The project is divided into small steps and each step is done in a separate cell of the notebook. The output of one step is used in the next step, as shown in Figure 2.

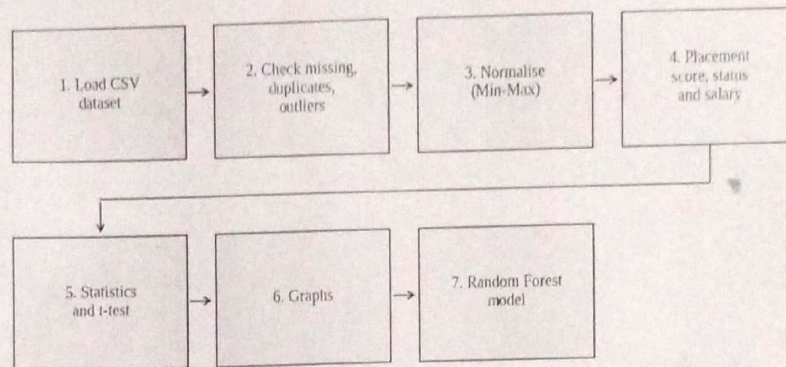


Figure 2: Workflow of the College Placement Data Analyzer

1. Load: read the CSV file and print its shape, first rows and data types.
2. Check data: find missing values, duplicate rows and CGPA outliers.
3. Normalize: scale four columns between 0 and 1.
4. Create labels: calculate composite score, placement status and salary.
5. Analyze: statistics, t-test and department wise percentages.
6. Visualize: draw and save six graphs.
7. Model: train Random Forest, check accuracy and feature importance.

6.2 Placement Score Rule

As the dataset has no actual placement result, we made our own rule based on what is usually seen in campus placements. First, a composite score out of 100 is calculated by giving different weights to the columns, as shown in Table 5. CGPA gets the highest weight, followed by aptitude and technical skills.

Table 5: Weights used in the composite score

Component	Calculation	Maximum marks
CGPA	$CGPA / 10 \times 35$	35
Aptitude score	$Aptitude / 100 \times 25$	25
Technical skills	$Technical / 10 \times 20$	20
Communication score	$Communication / 100 \times 15$	15
Internship	5 if Yes, otherwise 0	5
Total		100

A student is marked as Placed only if two conditions are satisfied together: (1) the student is eligible, which means CGPA is 6.0 or more and backlogs are 1 or less, and (2) the composite score is 60 or more. Otherwise the student is Not Placed. For placed students a rough salary is estimated as:

$$\text{Salary_LPA} = 3.5 + (CGPA - 6) * 1.2 + (Technical_Skills / 10) * 2.5 + Projects * 0.4$$

This salary is only an estimate made by a formula to show how such a column can be created, it is not real salary data.

7. Implementation

This chapter explains the code of the notebook step by step along with the output it produced.

7.1 Importing Libraries

```
import os
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.dpi'] = 120
```

7.2 Loading the Dataset

The CSV file is read using `pd.read_csv()`. A try-except block is used so that a clear message is shown if the file is not found in the folder.

```
filename = 'college_placement_dataset_500.csv'

try:
    df = pd.read_csv(filename)
    print("file loaded, shape:", df.shape)
except FileNotFoundError:
    print("Error: could not find", filename, "- make sure it's in the same folder")
    raise

print(df.head())
df.info()
```

Output:

```
file loaded, shape: (500, 12)
  Student_ID  Gender  Age  ...  Aptitude_Score  Communication_Score  Certifications
0   STU001    Male   22  ...                45                    77                3
1   STU002    Male   21  ...                66                    54                3
2   STU003    Male   21  ...                88                    61                0
3   STU004    Male   22  ...                47                    99                3
4   STU005  Female   22  ...                89                    58                0

[5 rows x 12 columns]
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Student_ID            500 non-null   str
 1   Gender                500 non-null   str
 2   Age                   500 non-null   int64
 3   Department            500 non-null   str
```

```

4   CGPA                500 non-null   float64
5   Backlogs            500 non-null   int64
6   Projects            500 non-null   int64
7   Internship          500 non-null   str
8   Technical_Skills    500 non-null   int64
9   Aptitude_Score      500 non-null   int64
10  Communication_Score 500 non-null   int64
11  Certifications      500 non-null   int64
dtypes: float64(1), int64(7), str(4)
memory usage: 47.0 KB

```

The output shows that the dataset has 500 rows and 12 columns, and every column has 500 non-null values, which already tells us there are no missing values.

7.3 Data Cleaning and Checking

Here we check the missing values and duplicate rows. For outliers the IQR rule is used on the CGPA column: any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is treated as an outlier.

```

print("\nmissing values:\n", df.isnull().sum())

dup = df.duplicated().sum()
print("duplicate rows:", dup)
if dup > 0:
    df = df.drop_duplicates()
    print("removed duplicates, new shape:", df.shape)

# check CGPA for outliers using IQR rule
q1 = df['CGPA'].quantile(0.25)
q3 = df['CGPA'].quantile(0.75)
iqr = q3 - q1
low = q1 - 1.5 * iqr
high = q3 + 1.5 * iqr
cgpa_outliers = df[(df['CGPA'] < low) | (df['CGPA'] > high)]
print(f"\nCGPA Q1={q1:.2f}, Q3={q3:.2f}, IQR={iqr:.2f}")
print(f"outlier range is below {low:.2f} or above {high:.2f}")
print("number of outliers found:", len(cgpa_outliers))

```

Output:

```

missing values:
Student_ID      0
Gender          0
Age             0
Department      0
CGPA            0
Backlogs        0
Projects        0
Internship      0
Technical_Skills 0
Aptitude_Score  0
Communication_Score 0
Certifications  0
dtype: int64
duplicate rows: 0

CGPA Q1=6.67, Q3=8.83, IQR=2.16
outlier range is below 3.43 or above 12.08
number of outliers found: 0

```

The result is that there are no missing values, no duplicate rows and no CGPA outliers. The CGPA values lie between 5.51 and 9.90, which is well inside the allowed range of 3.43 to 12.08, so no row had to be removed.

7.4 Normalization

Min-Max normalization changes the values of a column to the range 0 to 1 using the formula $(x - \min) / (\max - \min)$. We applied it on CGPA, Aptitude_Score, Technical_Skills and Communication_Score so that all of them are on the same scale.

```
scaler = MinMaxScaler()
cols_to_scale = ['CGPA', 'Aptitude_Score', 'Technical_Skills', 'Communication_Score']
scaled = scaler.fit_transform(df[cols_to_scale])
scaled_df = pd.DataFrame(scaled, columns=[c + '_scaled' for c in cols_to_scale])
print("\nnormalized sample:\n", scaled_df.head())
```

Output:

```
normalized sample:
   CGPA_scaled  ...  Communication_Score_scaled
0    0.637813  ...                0.616667
1    0.029613  ...                0.233333
2    0.587699  ...                0.350000
3    0.091116  ...                0.983333
4    0.077449  ...                0.300000
```

7.5 Placement Score, Status and Salary

The composite score, placement status and salary estimate are created here using the rule explained in Chapter 6. np.where() is used to apply the conditions on all rows at once.

```
intern_yes = df['Internship'].apply(lambda x: 1 if x == 'Yes' else 0)

score = (df['CGPA'] / 10 * 35) + (df['Aptitude_Score'] / 100 * 25) + \
        (df['Technical_Skills'] / 10 * 20) + (df['Communication_Score'] / 100 * 15) + \
        (intern_yes * 5)
df['Composite_Score'] = score.round(2)

eligible = (df['CGPA'] >= 6.0) & (df['Backlogs'] <= 1)
df['Placement_Status'] = np.where(eligible & (df['Composite_Score'] >= 60),
                                  'Placed', 'Not Placed')

# rough salary estimate just for placed students
df['Salary_LPA'] = np.where(
    df['Placement_Status'] == 'Placed',
    (3.5 + (df['CGPA'] - 6) * 1.2 + (df['Technical_Skills'] / 10) * 2.5
     + df['Projects'] * 0.4).round(2),
    0
)

print("\nplacement status counts:")
print(df['Placement_Status'].value_counts())
print(df['Placement_Status'].value_counts(normalize=True).round(4) * 100)
```

Output:

```

placement status counts:
Placement_Status
Placed      338
Not Placed   162
Name: count, dtype: int64
Placement_Status
Placed      67.6
Not Placed   32.4
Name: proportion, dtype: float64

```

Out of 500 students, 338 (67.6%) are Placed and 162 (32.4%) are Not Placed. A total of 393 students satisfied the eligibility condition (CGPA of 6.0 or more and at most one backlog), and 338 of them also reached the composite score of 60.

7.6 Statistics and Hypothesis Test

The skewness and kurtosis of CGPA tell about the shape of its distribution. A one sample t-test is used to check whether the average CGPA of the students is really different from 7.5. The hypotheses were: H_0 : mean CGPA = 7.5 and H_1 : mean CGPA is not equal to 7.5, tested at the 5% level of significance. The department wise placement percentage is also found.

```

print("\nCGPA skew:", round(stats.skew(df['CGPA']), 4))
print("CGPA kurtosis:", round(stats.kurtosis(df['CGPA']), 4))

t_stat, p_val = stats.ttest_1samp(df['CGPA'], popmean=7.5)
print(f"\nt-test vs 7.5 -> t = {t_stat:.4f}, p = {p_val:.6f}")
if p_val < 0.05:
    print("p < 0.05 so we reject H0, mean CGPA is significantly different from 7.5")
else:
    print("p >= 0.05, not enough evidence to say it's different from 7.5")

# department wise placement %
dept_table = df.groupby('Department')['Placement_Status'] \
    .value_counts(normalize=True).unstack().fillna(0) * 100
print("\nplacement % by department:\n", dept_table.round(2))

```

Output:

```

CGPA skew: -0.0069
CGPA kurtosis: -1.185

t-test vs 7.5 -> t = 4.4206, p = 0.000012
p < 0.05 so we reject H0, mean CGPA is significantly different from 7.5

placement % by department:
Placement_Status  Not Placed   Placed
Department
Civil              40.26    59.74
Computer           31.76    68.24
Electrical          32.14    67.86
Electronics         30.00    70.00
IT                  35.37    64.63
Mechanical          25.61    74.39

```

7.7 Data Visualization

Six graphs were drawn using Seaborn and saved as PNG files. The code of the first graph is shown below; the others are made in the same way with different plot functions.

```
plt.figure(figsize=(8, 5))
sns.histplot(df['CGPA'], kde=True, bins=20)
plt.title('Distribution of CGPA')
plt.xlabel('CGPA')
plt.tight_layout()
plt.savefig('fig1_cgpa_distribution.png')
plt.close()

sns.boxplot(x='Placement_Status', y='CGPA', data=df) # fig2
sns.countplot(x='Department', hue='Placement_Status', data=df) # fig3
sns.heatmap(df[num_cols].corr(), annot=True, fmt='.2f') # fig4
sns.scatterplot(x='Aptitude_Score', y='Communication_Score',
                hue='Placement_Status', data=df) # fig5
sns.countplot(x='Internship', hue='Placement_Status', data=df) # fig6

print("\nsaved 6 plots")
```

Output:

```
saved 6 plots
```

7.8 Random Forest Model

For the machine learning part, seven columns were taken as features (CGPA, Backlogs, Projects, Technical_Skills, Aptitude_Score, Communication_Score and Certifications) and Placement_Status was the target (1 for Placed, 0 for Not Placed). The data was divided into 80% training (400 rows) and 20% testing (100 rows) with stratification so that the placed and not placed ratio stays the same in both parts. A Random Forest with 100 trees and a maximum depth of 6 was used.

```
feature_cols = ['CGPA', 'Backlogs', 'Projects', 'Technical_Skills',
                'Aptitude_Score', 'Communication_Score', 'Certifications']
X = df[feature_cols]
y = df['Placement_Status'].apply(lambda x: 1 if x == 'Placed' else 0)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(f"\ntrain size: {len(X_train)}, test size: {len(X_test)}")

model = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=42)
model.fit(X_train, y_train)

pred = model.predict(X_test)
acc = accuracy_score(y_test, pred)
print(f"\naccuracy: {acc*100:.2f}%")
print(classification_report(y_test, pred, target_names=['Not Placed', 'Placed']))
print("confusion matrix:\n", confusion_matrix(y_test, pred))

imp = pd.Series(model.feature_importances_, index=feature_cols)
imp = imp.sort_values(ascending=False)
print("\nfeature importance:\n", imp.round(4))
```

Output:

```
train size: 400, test size: 100
```

```
accuracy: 97.00%
```

	precision	recall	f1-score	support
Not Placed	1.00	0.91	0.95	32
Placed	0.96	1.00	0.98	68
accuracy			0.97	100
macro avg	0.98	0.95	0.96	100
weighted avg	0.97	0.97	0.97	100

```
confusion matrix:
```

```
[[29  3]
 [ 0 68]]
```

```
feature importance:
```

CGPA	0.3905
Backlogs	0.2682
Technical_Skills	0.1036
Aptitude_Score	0.1033
Communication_Score	0.0760
Projects	0.0347
Certifications	0.0237

```
dtype: float64
```

8. Results and Discussion

8.1 CGPA Distribution

Figure 3 shows the distribution of CGPA. The curve is spread almost evenly between 5.5 and 10 and there is no single sharp peak. The skewness is -0.0069 , which is almost zero, so the distribution is symmetric. The kurtosis is -1.185 , which is negative, so the curve is flatter than a normal curve. This also matches the IQR check, where no outlier was found.

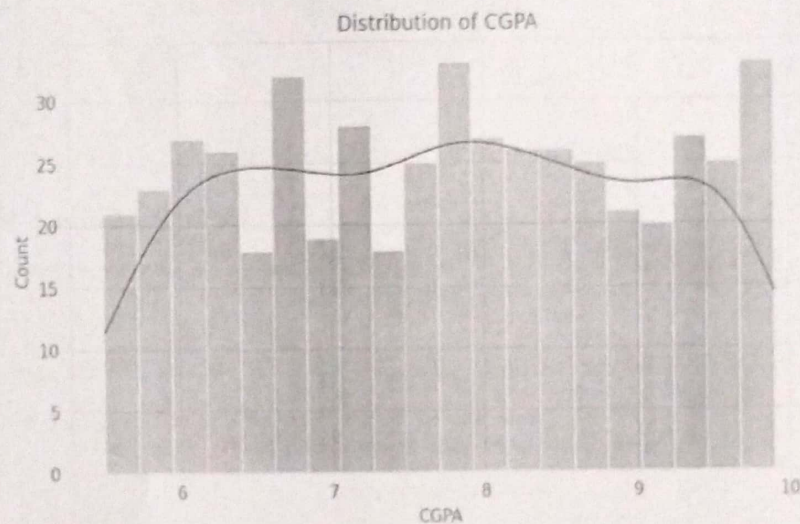


Figure 3: Distribution of CGPA

The t-test gave $t = 4.4206$ and $p = 0.000012$. Since p is much smaller than 0.05 , the null hypothesis is rejected, and we can say that the average CGPA of the students (7.75) is significantly different from 7.5 .

8.2 Placement Result

As per our placement rule, 338 students (67.6%) are placed and 162 students (32.4%) are not placed, as shown in Figure 4. The gender wise result is nearly the same: 68.27% of female students and 66.93% of male students are placed, so the rule does not favour any gender.

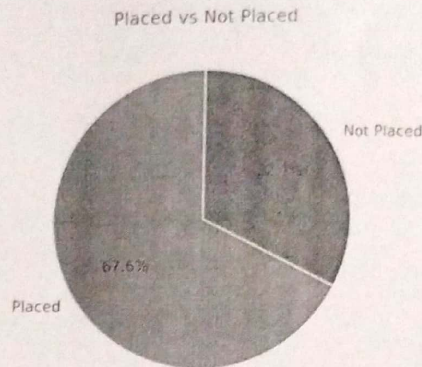


Figure 4: Placed and Not Placed students

8.3 CGPA and Placement

Figure 5 compares the CGPA of placed and not placed students. The average CGPA of placed students is 8.16 while for not placed students it is 6.91. The placed group is clearly higher, which is expected because CGPA has the highest weight in the composite score and a minimum CGPA of 6.0 is needed to be eligible.

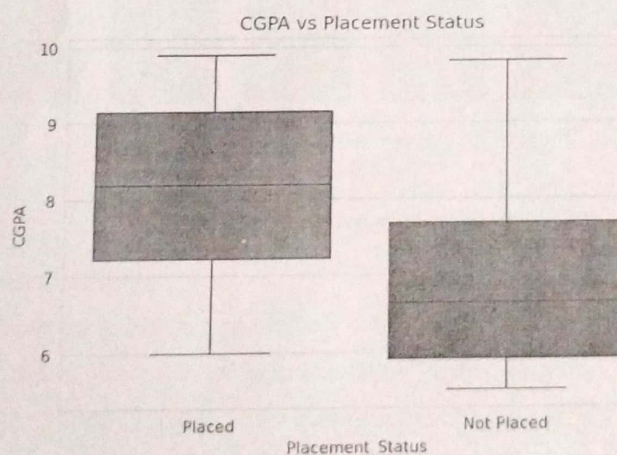


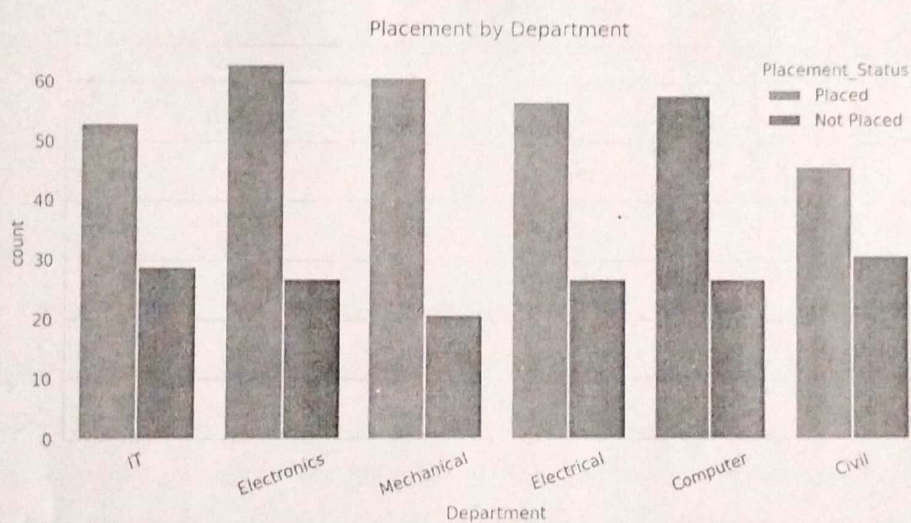
Figure 5: CGPA vs placement status

8.4 Department-wise Placement

Table 6 and Figure 6 show the placement result of each department. Mechanical has the highest placement percentage (74.39%) and Civil has the lowest (59.74%). The other four departments are between 64% and 70%. Since the students of every department were scored by the same rule, these differences come only from the CGPA, backlog and skill values of the students in that department.

Table 6: Department-wise placement

Department	Placed	Not Placed	Placed %
Civil	46	31	59.74
Computer	58	27	68.24
Electrical	57	27	67.86
Electronics	63	27	70.00
IT	53	29	64.63
Mechanical	61	21	74.39
Total	338	162	67.60

*Figure 6: Placement by department*

8.5 Correlation Analysis

The correlation heatmap in Figure 7 shows how the numeric columns are related to each other. The composite score has the strongest relation with Technical_Skills (0.61), then Aptitude_Score (0.51) and CGPA (0.50), which is natural because these three have the biggest weights in the formula. Salary_LPA is also positively related to the composite score (0.59). The input columns such as Age, Projects and Certifications show almost no relation with each other or with the score, so the data does not have any redundant column.

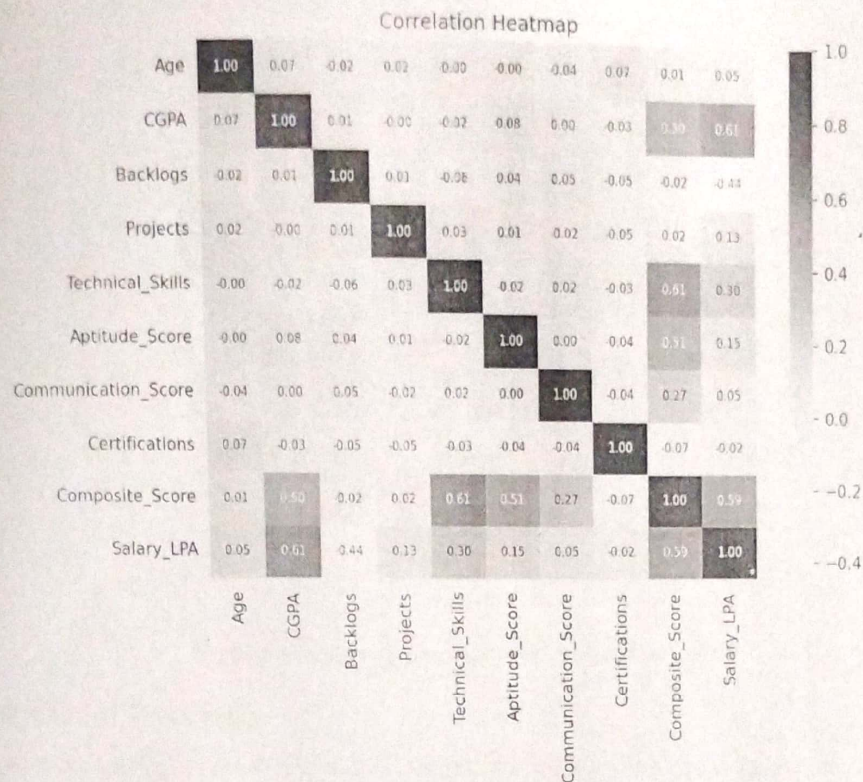


Figure 7: Correlation heatmap

8.6 Aptitude and Communication

Figure 8 is a scatter plot of aptitude score and communication score, where each dot is a student. The two scores are spread over the full range of 40 to 100 and do not show any straight line pattern. Placed students are found mostly in the upper right side, where both scores are higher, but there are also placed students with a low communication score if their CGPA and technical skills are good.

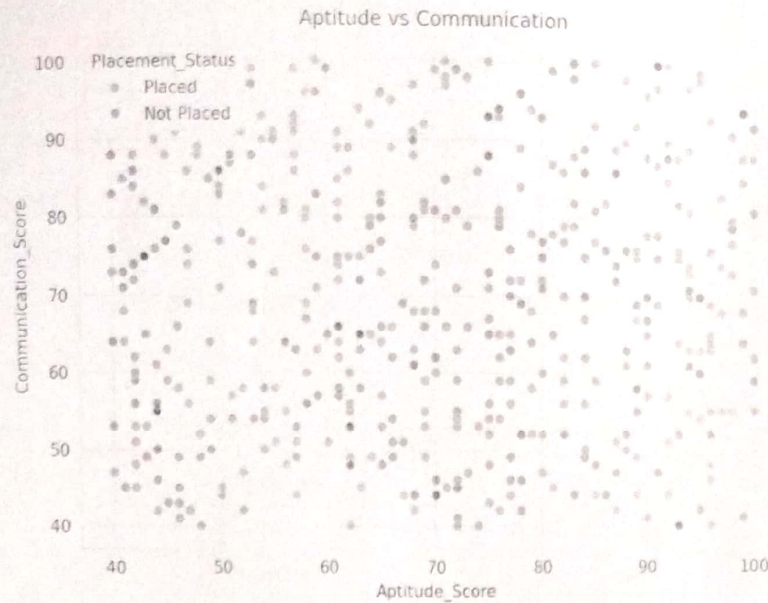


Figure 8: Aptitude score vs communication score

8.7 Effect of Internship

Figure 9 shows that students who did an internship have a better placement chance. 75.62% of the students with an internship are placed, compared to 60.08% of those without an internship, a difference of around 15 percentage points. Part of this comes from the 5 marks directly given for internship in the score.

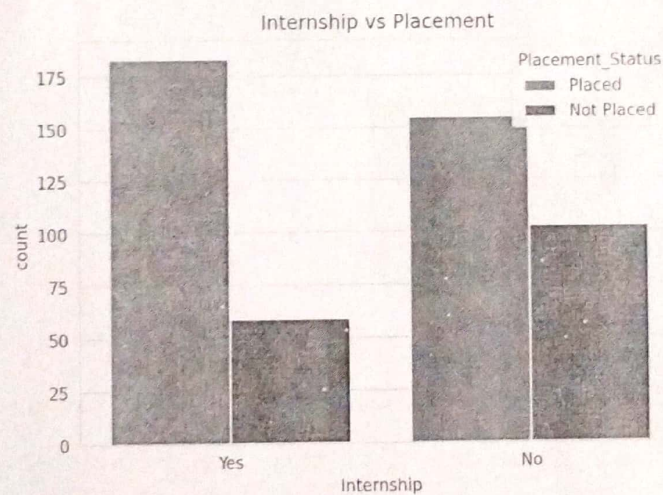


Figure 9: Internship vs placement

8.8 Salary Estimate

For the 338 placed students the estimated salary has a mean of 8.69 LPA, a minimum of 4.57 LPA, a maximum of 11.99 LPA and a standard deviation of 1.58. The department wise average is almost the same everywhere (between 8.41 LPA for Electrical and 8.83 LPA for IT), because

the formula only uses CGPA, technical skills and projects. As said before, this is an estimate and not real salary data.

8.9 Model Performance

The Random Forest model gave an accuracy of 97% on the 100 test students. Table 7 gives the precision, recall and F1-score and Figure 10 shows the confusion matrix. Out of 68 placed students all 68 were predicted correctly, and out of 32 not placed students 29 were correct and 3 were wrongly predicted as placed.

Table 7: Classification report on test data

Class	Precision	Recall	F1-score	Support
Not Placed	1.00	0.91	0.95	32
Placed	0.96	1.00	0.98	68
Accuracy			0.97	100
Macro average	0.98	0.95	0.96	100
Weighted average	0.97	0.97	0.97	100

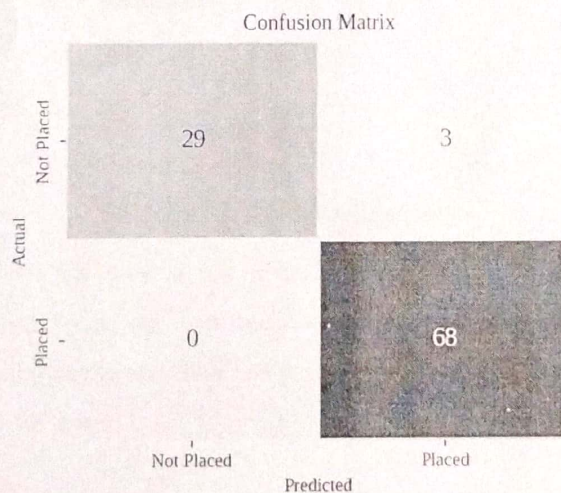
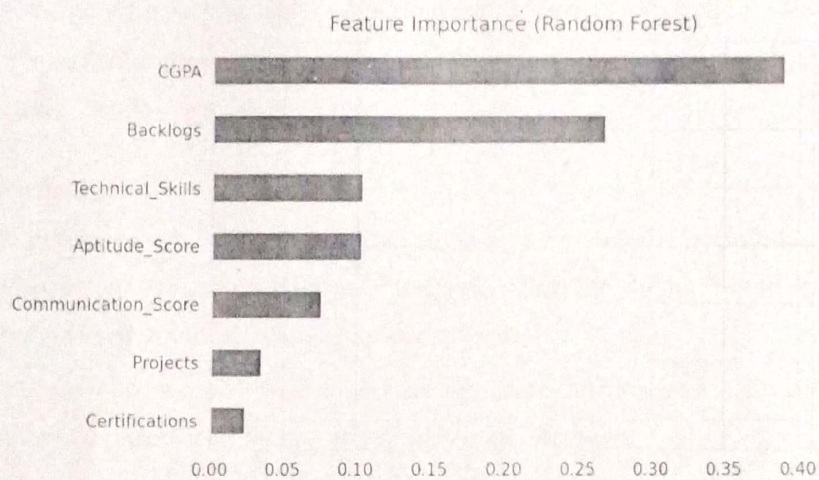


Figure 10: Confusion matrix of the Random Forest model

The feature importance in Table 8 and Figure 11 tells which columns the model used the most. CGPA (0.39) and Backlogs (0.27) are the top two, which agrees with the eligibility condition of our rule. Technical skills and aptitude score come next with about 0.10 each, and Projects and Certifications have the lowest importance.

Table 8: Feature importance

Feature	Importance
CGPA	0.3905
Backlogs	0.2682
Technical_Skills	0.1036
Aptitude_Score	0.1033
Communication_Score	0.0760
Projects	0.0347
Certifications	0.0237

*Figure 11: Feature importance*

Important note: the 97% accuracy should be read carefully. The placement status was created from the same columns (using our own rule), so the model is mostly learning back that rule and not predicting real placements. With real placement records from the college the accuracy would be expected to be lower.

9. Conclusion, Limitations and Future Scope

9.1 Conclusion

In this micro project we made a College Placement Data Analyzer in Python which covers the whole data science process on a dataset of 500 students. The data was checked and found to be clean, with no missing values, duplicates or CGPA outliers. We created a composite score, placement status and salary estimate, which showed that 67.6% of the students are placed. Statistical analysis showed that the mean CGPA (7.75) is significantly different from 7.5, and that the Mechanical and Electronics departments have higher placement rates while Civil has the lowest. The graphs showed that CGPA, backlogs, technical skills, aptitude and internship experience are the main factors, and the Random Forest model confirmed the same with 97% accuracy. Through this project we also learned how to use Pandas, NumPy, Seaborn, SciPy and Scikit-learn together in a practical way.

9.2 Limitations

- The dataset has no real placement column, so the placement status and salary were created by our own rule and formula. Because of this, the analysis shows the behaviour of the rule and not of real campus placements.
- The model accuracy of 97% is high mainly because the label was made from the same features, so it cannot be treated as real prediction accuracy.
- Only the CGPA column was checked for outliers, and the dataset is small (500 rows).
- The salary column is only an estimate and does not consider company, role or location.
- The normalized columns were created for demonstration and were not used further in the model.

9.3 Future Scope

- Use real placement records of the college (placed / not placed and actual package) to train and test the model.
- Try other models such as Logistic Regression, Decision Tree and XGBoost and compare them using cross validation.
- Add more factors like 10th and 12th marks, coding platform ratings, and company wise data.
- Make a small web dashboard using Streamlit or Flask where a student can enter details and see the chance of placement.
- Add outlier checks for all numeric columns and automatic report generation.

References

1. Pandas Documentation, <https://pandas.pydata.org/docs/>
2. NumPy Documentation, <https://numpy.org/doc/>
3. Matplotlib Documentation, <https://matplotlib.org/stable/>
4. Seaborn Documentation, <https://seaborn.pydata.org/>
5. SciPy Documentation (scipy.stats), <https://docs.scipy.org/doc/scipy/>
6. Scikit-learn Documentation, <https://scikit-learn.org/stable/>
7. Python for Data Science (BE05000231) course material, Gujarat Technological University.